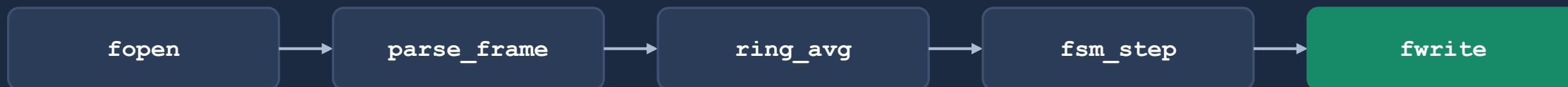


파일 I/O · 디버깅 · 종합 프로젝트

새 문법은 거의 없다. 어제까지의 것들을 파일과 디버거로 묶어 4일치를 한 프로그램으로



오늘의 8교시

오전은 파일(열기·텍스트·바이너리·링 버퍼), 오후는 도구(gdb·구조)와 프로젝트

1

파일 열고 닫기

FILE*, 모드, errno, fclose

2

텍스트·CSV

fprintf, 버퍼링, fflush

3

바이너리 I/O

fwrite/fread, fseek/ftell

4

링 버퍼

head/tail/count, 이동평균

5

디버깅

gdb 5명령, assert, ASan

6

코드 품질

모듈, 에러 규약, make test

7

프로젝트 ①

센서 데이터 수집기

8

프로젝트 ② · 정리

마무리, 4일 지도, 평가 예고

fopen → 검사 → 사용 → fclose

1교시

```
FILE *fp = fopen("temps.txt", "r");
if (fp == NULL) { perror("fopen"); return 1; }

char line[64];
while (fgets(line, sizeof line, fp)) { // stdin 자리에 fp
    line[strcspn(line, "\n")] = '\0';
    ...
}
fclose(fp); // malloc/free 의 짝
```

stdin·stdout·stderr 도 미리 열린 FILE*. 어제 코드가 그대로 파일에 쓰인다.

"r"	읽기	없으면 NULL
"w"	쓰기	있으면 내용 삭제 !
"a"	덧붙이기	로그는 이것
"r+"	읽기+쓰기	내용 유지
"rb" "wb"	바이너리	3교시

⚠ errno 는 실패 직후 변수에 복사. printf 가 덮어쓴다.
⚠ fclose 없이 2000번 열면 1021번째에서 "Too many open files".

fprintf 는 버퍼에 쓴다 — fflush 가 디스크에 쓴다

2교시

```
FILE *out = fopen("log.csv", "w");
fprintf(out, "seq,temp,state\n");
fprintf(out, "%d,%.1f,%s\n", 1, 25.3, "IDLE"); // printf +
파일
fputs("2,17.0,HEAT\n", out);
```

```
// 여기서 죽으면? → 파일은 비어 있다 (4KB 버퍼)
fflush(out); // 지금 디스크에
fclose(out); // flush + 닫기
```

```
printf(...) == fprintf(stdout, ...)
fprintf(stderr, ...) // ./app > out.csv 해도 화면에
```

fflush / fclose
/ 버퍼 가득

버퍼 (메모리)
T=25℃ T=26℃

디스크
log.csv

로그가 중요하면 줄마다 fflush. 전원이 나가도 마지막 줄까지 남는다. 7교시 events.log 가 이 방식.

```
// CSV 읽기 = Day 2 파싱 그대로
snprintf(buf, sizeof buf, "%s", line); // strtok 은 원본
을 고침
a = strtok(buf, ","); b = strtok(NULL, ","); ...
seq = strtol(a, &end, 10); if (end == a) return 0;
t = strtod(b, &end); if (end == b) return 0;
```

CSV: 사람·Excel 이 읽는다. 대신 크고, 파싱 필요. → 3교시 바이너리

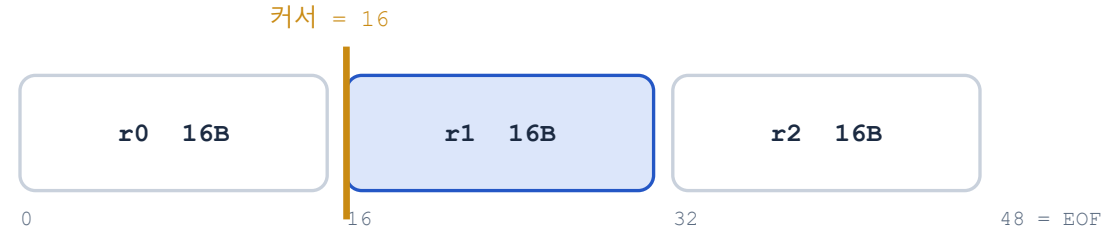
바이너리 — 구조체 통째로, 커서는 fseek 로

3교시

```
Reading r[3] = {{25.3,60,120}, {17.0,55,118}, {36.9,70,150}};

FILE *fp = fopen("log.bin", "wb");
fwrite(r, sizeof r[0], 3, fp); // 주소, 크기, 개수, 파일 — qsort
                                모양
fclose(fp);                      // 48 bytes

fp = fopen("log.bin", "rb");
fseek(fp, 0, SEEK_END); long size = ftell(fp); // 48
fseek(fp, 1 * sizeof(Reading), SEEK_SET);      // 2번째로
Reading x;
if (fread(&x, sizeof x, 1, fp) == 1) ...      // 반환값 검사!
```



```
$ od -A d -t x1 log.bin | head -1
00000000 cd cc cc cc cc 4c 39 40 3c 00 00 00 78 00 00 00
           └ double 25.3 (LE) ─┘ └ 60 ┘ └ 120 ┘
```

⚠ 같은 컴파일러·같은 CPU 에서만. 다른 장치엔 패딩(Day 3)·엔디안(Day 1)이 다르다 → 필드별 바이트 직렬화.
읽기↔쓰기 전환 사이엔 fseek 필수. "w+b" 로 열면 파일이 지워진다 — "r+b".

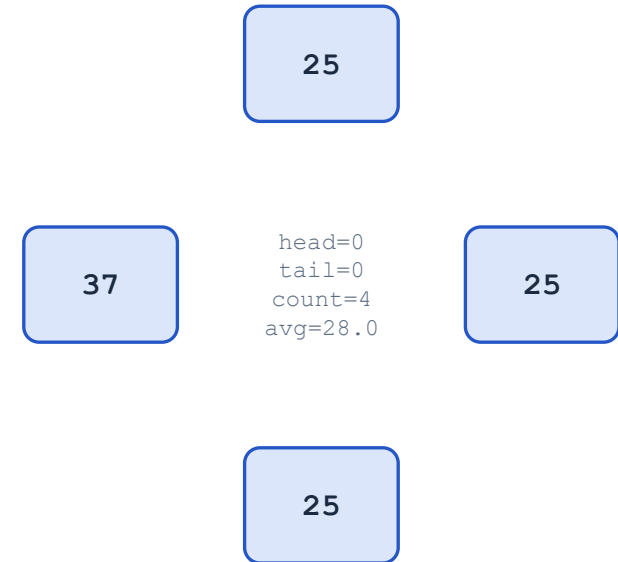
링 버퍼 — 고정 메모리로 흐르는 데이터 받기

4교시

```
typedef struct { double buf[RING_CAP]; int head, tail, count; } Ring;

void ring_push(Ring *r, double v) {
    if (r->count == RING_CAP) {          // 가득: 가장 오래된 것 버림
        r->tail = (r->tail + 1) % RING_CAP; r->count--;
    }
    r->buf[r->head] = v;
    r->head = (r->head + 1) % RING_CAP;   // 끝에서 0 으로
    r->count++;
}

double ring_avg(const Ring *r) {        // tail 부터 count 개
    for (i = 0; i < r->count; i++) s += r->buf[(r->tail + i) %
RING_CAP];
}
```



왜 링인가: 센서 100Hz, 로그 1Hz. realloc 은 MCU 에서 위험, 배열 당기기는 느림. 고정 배열 원형 = 할당도 이동도 없다.
△ count 없이 head/tail 만 쓰면 빈 것과 가득 찬 것을 구분 못 한다. 인터럽트↔메인 사이엔 volatile (Day 3).

📺 D4-4 링 버퍼 — 시나리오 재생

gdb — Day 3 의 조용한 버그를 실행 중에 본다

5교시

컴파일 경고 없음 · 죽지 않음 · 출력도 그럴듯함 → "HEAT 에서 무엇이 불리나" 를 보는 수밖에

```
$ gcc -Wall -Wextra -g -O0 fsm_bug.c -o fsm_bug
$ gdb ./fsm_bug
(gdb) break fsm_step
(gdb) run < temps.txt
Breakpoint 1, fsm_step (cur=ST_IDLE, t=25) at fsm.c:17
(gdb) print TABLE[cur]
$1 = (Handler) 0x...1a9 <on_idle>
(gdb) continue
Breakpoint 1, fsm_step (cur=ST_HEAT, t=19) at fsm.c:17
(gdb) print TABLE[cur]
$2 = (Handler) 0x...1f2 <on_cool>      ← !!! HEAT 인데 on_cool
(gdb) bt
#0  fsm_step (cur=ST_HEAT, t=19) at fsm.c:17
#1  0x... in main () at main.c:31
```

break 위치	함수명 또는 파일:줄	b
run [인자]	실행 (< 리다이렉션 OK)	r
next / step	한 줄 (건너뛸 / 안으로)	n / s
print 식	값. p/x 16진, p *p, p a[0]@5	p
bt	스택 — 세그폴트 직후 1순위	
watch 변수	값이 바뀌면 멈춤	
continue / quit	다음 중단점 / 종료	c / q

🎬 D4-5 gdb 스텝 시뮬레이터 — break → run → continue → print

assert · ASan — 버그를 일찍 터뜨린다

5교시

```
#include <assert.h>
double median(const int *a, int n) {
    assert(n > 0 && "median of empty array"); // 계약
    return n % 2 ? a[n/2] : (a[n/2-1] + a[n/2]) / 2.0;
}
median(a, 0);
// as.c:5: median: Assertion `n > 0 && "..."' failed. Aborted

$ gcc -DNDEBUG as.c      # assert 사라짐 → -795798151.5 (쓰레기)
```

```
int a[3] = {1, 2, 3}; int i = 3;
printf("%d\n", a[i]);           // 경고 없음, Day 2 에선
                                조용히 통과
```

```
$ gcc -g -fsanitize=address san.c -o san && ./san
==509==ERROR: AddressSanitizer: stack-buffer-overflow
#0 0x... in main san.c:2
READ of size 4 ... 0 bytes after 12-byte region
```

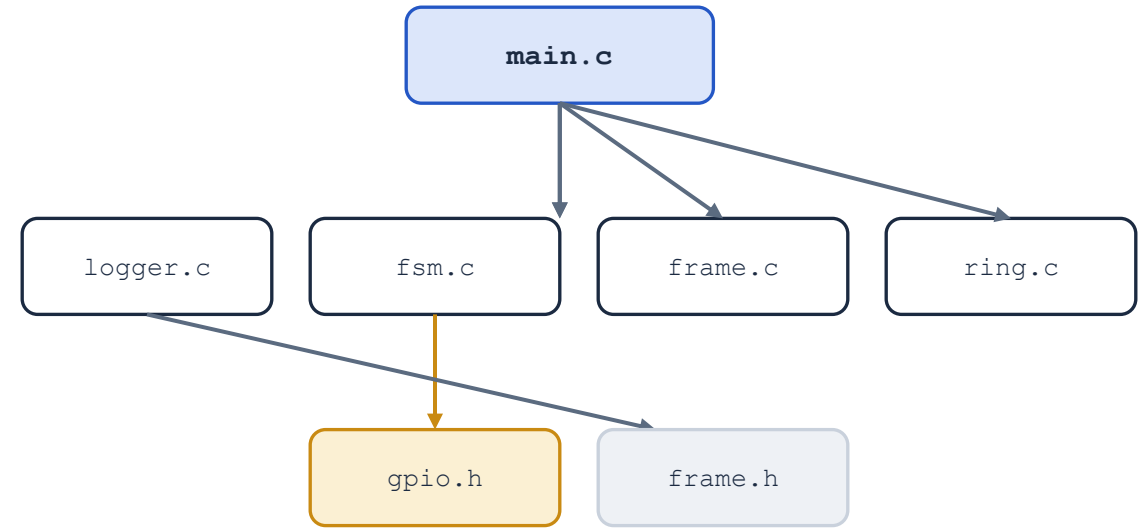
assert 는 "이렇게 부르면 안 된다" 는 계약. NDEBUG 로 제거되므로 부수효과 있는 식 금지 — assert(fp = fopen(...)) 는 릴리스에서 fopen 자체가 사라진다.

ASan: 배열 밖·해제 후 사용·힙 오버플로·누수를 실행 즉시 줄 번호와 함께. valgrind 보다 빠름(2~3배 느려짐). 개발 중 켜고 릴리스에서 뺀다 → make asan.

모듈은 아래로만 의존한다 — 4일치 코드의 구조

6교시

```
sensor_logger/  
├─ frame.h / frame.c    Day 2  parse_frame  
├─ ring.h / ring.c      Day 4  ring_push/pop/avg  
├─ fsm.h / fsm.c        Day 3  fsm_step  
├─ gpio.h               Day 3  GPIO_SET/CLR  
├─ logger.h / logger.c  Day 4  log_open/row/close  
├─ main.c               조립만  
├─ test_ring.c          단위 테스트  
└─ Makefile             test · asan · debug
```



ring 은 아무것도 모른다 → 단독 테스트, 다른 프로젝트로 그대로

① 헤더엔 남이 알아야 할 것만

구조체·원형·매크로, 세부는 .c 의 static

② 의존은 한 방향

ring.c 는 frame.h 를 모른다

③ main 에 로직 없음

모듈 함수를 부르는 순서만

④ 에러 규약은 하나로

성공1/실패0 · 0/음수코드 · 포인터/NULL — 섞지 않기 (ESP32: esp_err_t)

📁 D4-6 모듈 의존 그래프 — 나쁜 예 토글

Makefile — test · asan · debug 타킷

6교시

```
CC      = gcc
CFLAGS  = -Wall -Wextra -g -O0
OBJS     = main.o frame.o ring.o fsm.o logger.o

logger: $(OBJS)
        $(CC) $(CFLAGS) -o $@ $^          # $@ 타킷 $^ 의존 전체
test_ring: test_ring.o ring.o
        $(CC) $(CFLAGS) -o $@ $^
test: test_ring
        ./test_ring
asan: CFLAGS += -fsanitize=address    # 타킷별 변수
asan: clean logger
debug: logger
        gdb -batch -ex run -ex bt --args ./logger temps.txt
%.o: %.c
        $(CC) $(CFLAGS) -c $<
main.o: frame.h ring.h fsm.h gpio.h logger.h
.PHONY: test asan debug clean
```

```
#include <assert.h>
#include "ring.h"
int main(void) {
    Ring r; ring_init(&r); double v;
    assert(ring_pop(&r, &v) == false);
    for (i = 0; i < RING_CAP + 2; i++)
        ring_push(&r, i);
    assert(r.count == RING_CAP);
    assert(ring_pop(&r, &v) && v == 2);
    printf("test_ring: OK\n");
}
```

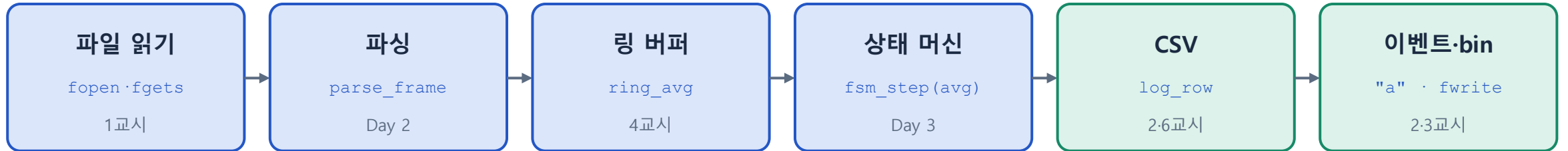
make test 가 통과하지 않으면 main 을 건드리지 않는다.
make asan → 메모리 경찰 빌드. make debug → 죽으면 스택.

\$< 첫 의존. .PHONY 는 파일이 아닌 명령 타킷 표시
— clean 파일이 있어도 실행되게.

종합 프로젝트 — 센서 데이터 수집기 logger

7교시

`./logger temps.txt → readings.csv ("w") · events.log ("a") · readings.bin (fwrite)`



```
$ make && ./logger temps.txt
rows=10 bad=0 events=1
IDLE 6
HEAT 0 ← 17, 19 가 와도 4개 평균은 18 밑으로 안 감
COOL 4
ALERT 0 ← 36 한 번으론 평균이 35 를 못 넘음
$ cat events.log
7,29.75,IDLE,COOL
$ ls -l readings.bin → 160 = 10 × 16
```

Day 3 컨트롤러는 같은 입력에서 HEAT 2·ALERT 2. 이동평균 필터가 튀는 값을 무시한 결과 — 오경보 없음 vs 낮은 반응. RING_CAP=2 로 실험(8교시).
채울 곳: `realloc` 누적 · 마지막 `fwrite` · `log_open` 모드 인자. 조기 `return` 마다 `fclose/free`.

제출 체크리스트 · 마무리 순서

8교시



make clean && make && make test

경고 0 · test_ring: OK



make asan && ./logger temps.txt

ASan 에러 0



./logger / ./logger nope.txt

usage / perror + 종료 코드 1



events.log "a" · readings.csv "w"

두 번 실행: 이어짐 / 새로



readings.bin = rows × 16

load_bin 으로 다시 읽힘



worksheet_day4.html

8교시까지 채점 → 결과 코드

리뷰 실험 (30분)

- RING_CAP 을 2 로 → HEAT·ALERT 전이가 돌아오는가
- 빈 파일 · BAD 만 있는 파일 · 100줄 파일에서 죽지 않는가
- readings.bin 을 od 로 열어 16바이트 단위 확인
- make debug 로 일부러 세그폴트를 만들고 bt 읽기

미완성이어도 링 버퍼 + 파일 로깅 두 모듈이 붙어 있으면 통과. 제출: c_day4/sensor_logger/ + make test·asan 캡처.

4일의 여섯 문장

Day 1

컴파일은 4단계, 선언은 .h 정의는 .c, 자료형은 바이트 수와 해석 규칙, -Wall -Wextra 는 항상

Day 2

& 는 주소 * 는 값, 배열은 넘기면 포인터, 문자열은 \0 까지, malloc → 검사 → free

Day 3

구조체는 통째 복사, 비트는 |= &=~ ^=, 매크로는 텍스트, 함수도 주소, 상태 = 인덱스

Day 4

fopen → 검사 → fclose, 텍스트는 사람·바이너리는 기계, 링 버퍼는 고정 메모리, gdb·assert·ASan

임베디드로

고정폭 타입, 바이트 조립, volatile 레지스터, 콜백, 링 버퍼, 반환값 에러 규약

다음 주

바뀌는 건 GPIO_OUT 한 줄과 main 이 loop() 가 되는 것뿐

다음

9/14 파트 평가 · 9/22 MCU 동작 원리

9/14 (월) 파트 1 평가

필답: 4일 워크시트 범위 · 실습: 오늘 프로젝트 변형. 복습 퀴즈 4종을 먼저 다시 풀 것

9/22 (월) MCU 동작 원리 — ESP32

gpio.h 의 extern uint32_t GPIO_OUT 을 실제 주소로 바꾸는 것에서 시작

가져갈 파일

frame · ring · fsm · gpio · logger 모듈 그대로. main 만 setup()/loop() 로

제출: c_day4/sensor_logger/ + make test-asan 캡처 + worksheet_day4.html 결과 · 4일 수고하셨습니다